

Why We Scale by $\sqrt{d_k}$.

The one “magic constant” in attention is not magic at all — it is derived. We show, with explicit variance algebra and a worked numerical demonstration, why the dot products are divided by $\sqrt{d_k}$, what breaks without it, and why $\sqrt{d_k}$ (not d_k) is the right divisor.

THE CLAIM

If $\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$, $\text{Var}(q_i) = \text{Var}(k_i) = 1$, then $\text{Var}(q \cdot k) = d_k$.

So divide by $\sqrt{d_k}$ to restore unit variance: $\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = 1$.

Setup. Scores grow like $\sqrt{d_k}$; softmax saturates when its inputs are large; saturated softmax has vanishing gradients. The divisor keeps training alive.

Topic 2 of 5 in this module · companion to the M0.1 course page · v1.0

Contents

1 The problem: scores grow with dimension	2
2 The derivation: $\text{Var}(q \cdot k) = d_k$	2
3 Why large scores are fatal: softmax saturation	3
4 Worked demonstration — the same scores, two dimensions	3
5 Why $\sqrt{d_k}$ and not d_k	4
6 Summary	5
A Reproducible (NumPy)	5

1 The problem: scores grow with dimension

Recall step 2 of scaled dot-product attention (Topic 1): after forming the raw scores $S = QK^\top$, we divide by $\sqrt{d_k}$ before the softmax,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

A single score is a dot product of a query row q and a key row k , each of length d_k :

$$s = q \cdot k = \sum_{i=1}^{d_k} q_i k_i.$$

The dot product is a *sum of d_k terms*. As d_k grows, the sum has more terms, so its typical magnitude grows too. The scaling exists to cancel that growth.

Adding more dimensions to q and k should not, by itself, make one key “look” more relevant than another — it only adds more evidence. But a longer sum of random $\pm$ terms drifts further from zero. Without correction, a 64-dimensional model would produce scores eight times larger than a 1-dimensional one, purely as an artifact of dimension. The softmax would then treat those inflated gaps as enormous confidence. Scaling removes the artifact.

2 The derivation: $\text{Var}(q \cdot k) = d_k$

Assume the components of q and k are independent, zero-mean, unit-variance random variables (the standard assumption right after initialization, where weights are sampled this way):

$$\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0, \quad \text{Var}(q_i) = \text{Var}(k_i) = 1, \quad q_i \perp k_i.$$

Mean of one term. Because q_i and k_i are independent with zero mean,

$$\mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \mathbb{E}[k_i] = 0.$$

Variance of one term. Using $\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$ and independence,

$$\text{Var}(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - 0 = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = (\text{Var}(q_i))(\text{Var}(k_i)) = 1 \cdot 1 = 1.$$

Variance of the sum. The d_k product terms are mutually independent, so variances add:

$$\text{Var}(s) = \text{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} 1 = \boxed{d_k}.$$

The standard deviation is therefore $\sqrt{d_k}$: a typical score sits at scale $\pm\sqrt{d_k}$.

$$\text{Var}\left(\frac{s}{\sqrt{d_k}}\right) = \frac{\text{Var}(s)}{(\sqrt{d_k})^2} = \frac{d_k}{d_k} = 1. \quad (2)$$

Dividing by $\sqrt{d_k}$ rescales the standard deviation back to 1, independent of dimension. *That* is the whole reason for the constant.

Variance scales with d_k ; standard deviation scales with $\sqrt{d_k}$. The softmax cares about the *size* (standard deviation) of the score gaps, not their variance, so we divide by $\sqrt{d_k}$ — the standard deviation — not by d_k . Dividing by d_k would over-correct, shrinking the gaps toward zero and flattening the softmax into a near-uniform average that cannot discriminate (Section 5).

3 Why large scores are fatal: softmax saturation

Softmax of a vector z is $A_i = e^{z_i} / \sum_j e^{z_j}$. When the gaps between entries are large, the exponential of the largest entry dwarfs the rest and the output collapses toward a one-hot vector. In that regime the Jacobian (Topic 3) is

$$\frac{\partial A_i}{\partial z_j} = A_i(\delta_{ij} - A_j) \xrightarrow{A \rightarrow \text{one-hot}} 0,$$

because each A_i is either ≈ 0 or ≈ 1 , and $A_i(1 - A_i) \rightarrow 0$ in both cases. **A saturated softmax has vanishing gradients** — the model stops learning what to attend to. Keeping the scores at unit scale keeps the softmax in the smooth, high-gradient regime where $A_i(1 - A_i)$ is large.

4 Worked demonstration — the same scores, two dimensions

Take a fixed set of “unit-scale” similarities for one query against four keys, $b = [0.6, -0.2, 0.9, -0.3]$. In a real model of width d_k , the actual raw scores have standard deviation $\sqrt{d_k}$, i.e. they look like $b \cdot \sqrt{d_k}$. We compare $d_k = 4$ and $d_k = 64$, with and without the divisor.

Case	softmax weights	max weight
$d_k = 4$, raw scores $[1.2, -0.4, 1.8, -0.6]$	$[0.314, 0.063, 0.571, 0.052]$	0.571
$d_k = 4$, scaled ($\div 2$)	$[0.312, 0.140, 0.421, 0.127]$	0.421
$d_k = 64$, raw scores $[4.8, -1.6, 7.2, -2.4]$	$[0.083, 0.0001, 0.917, 0.0001]$	0.917
$d_k = 64$, scaled ($\div 8$)	$[0.312, 0.140, 0.421, 0.127]$	0.421

Two things to read off this table:

1. **Without scaling, dimension alone causes saturation.** At $d_k = 64$ the unscaled softmax puts 91.7% of its mass on one key and $\approx 0.01\%$ on two others — effectively a hard arg max with near-zero gradient to the losers. The underlying similarities b are identical to the $d_k = 4$ case; only the dimension changed.
2. **With scaling, the distribution is invariant to dimension.** Both scaled rows are exactly $[0.312, 0.140, 0.421, 0.127]$. Dividing by $\sqrt{d_k}$ exactly undoes the $\sqrt{d_k}$ inflation, so a model behaves the same at width 4 or width 64. That is the property we want.

This is why the scaling is sometimes called *temperature* $\tau = \sqrt{d_k}$: $\text{softmax}(z/\tau)$ with larger τ is “cooler” (flatter), smaller τ is “hotter” (peakier). Attention fixes $\tau = \sqrt{d_k}$ so the temperature self-adjusts to the model width.

5 Why $\sqrt{d_k}$ and not d_k

A natural question (and a common exam trap): if the variance is d_k , why not divide by d_k ? Because the softmax responds to the *standard deviation* of the inputs, which is $\sqrt{d_k}$.

Divisor	Resulting std of scores	Softmax behaviour
none	$\sqrt{d_k}$ (grows)	saturates as $d_k \uparrow$ — vanishing gradients
$\sqrt{d_k}$	1 (constant)	smooth, discriminative, dimension-stable
d_k	$1/\sqrt{d_k}$ (shrinks)	over-flattened — near-uniform, loses discrimination

Dividing by d_k would make scores shrink toward 0 as the model grows, pushing the softmax toward the uniform distribution $[\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}]$ regardless of content — a different failure mode, but just as useless. Only $\sqrt{d_k}$ holds the standard deviation at 1.

d_k is the *per-head* dimension. In multi-head attention with $d_{\text{model}} = 768$ and $h = 12$ heads, $d_k = 768/12 = 64$, so the divisor is $\sqrt{64} = 8$, **not** $\sqrt{768} \approx 27.7$. Using $\sqrt{d_{\text{model}}}$ over-scales every head and silently flattens attention. (This is the same point made in Topic 1’s pitfall and Topic 4.)

6 Summary

1. A score is a sum of d_k independent unit-variance products, so $\text{Var}(s) = d_k$ and $\text{std}(s) = \sqrt{d_k}$.
2. Large-magnitude scores saturate the softmax; a saturated softmax has $A_i(1 - A_i) \rightarrow 0$, i.e. vanishing gradients — learning stalls.
3. Dividing by $\sqrt{d_k}$ restores unit variance (Eq. 2), making attention's sharpness independent of model width.
4. The divisor is $\sqrt{d_k}$ (the standard deviation), not d_k (the variance), and it uses the *per-head* dimension.

A Reproducible (NumPy)

```
import numpy as np
def softmax(z):
    e = np.exp(z - z.max()); return e / e.sum()
b = np.array([0.6, -0.2, 0.9, -0.3])    # fixed unit-scale similarities
for dk in (4, 64):
    raw = b * np.sqrt(dk)                # real scores have std ~ sqrt(dk)
    print(dk, softmax(raw), softmax(raw / np.sqrt(dk)))
# 4  [0.314 0.063 0.571 0.052]  [0.312 0.140 0.421 0.127]
# 64 [0.083 0.000 0.917 0.000]  [0.312 0.140 0.421 0.127]  <- scaled is dim-invariant
```